

Hackathon Architecture & Defense Guide

Complete Technical Blueprint, "Why This / Why Not That" Matrix, AI Vocabulary & 12-Hour Action Plan

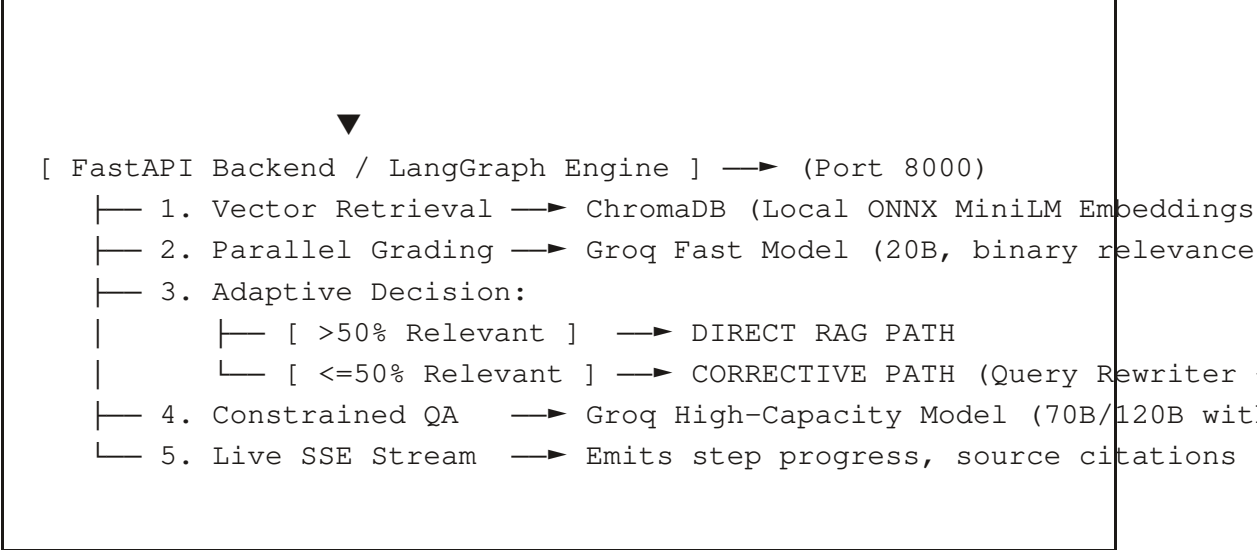
1. The 30-Second Elevator Pitch

"Most generative AI tools suffer from two critical flaws: **hallucinations** and **black-box opacity**. In high-stakes domains, an AI cannot simply guess—it must prove every statement.

Our platform solves this with an **Agentic Corrective RAG (CRAG)** engine powered by **LangGraph** and **FastAPI**, paired with an **editorial React presentation layer**. Unlike naive RAG systems that blindly trust whatever chunks they retrieve, our system acts as a **quality gatekeeper**: it evaluates retrieved evidence in parallel, filters irrelevant noise, triggers live web verification if internal knowledge is incomplete, and streams a live execution trace to the user with exact document and page-level citations."

2. System Architecture & Data Flow

```
[ User Browser / React 18 + Vite ] → (Port 5173 UI)
|
| Server-Sent Events (SSE) /ask/stream
```



3. Core Architectural Choices ("Why This, Why Not That?")

Decision	What We Chose	What We Rejected	Winning Rationale for Judges
Backend Layer	FastAPI (Python)	Pure Node.js / Express	The entire AI ecosystem (LangGraph, ChromaDB, PyMuPDF, ONNX) runs natively in Python. Running Python child-processes in Node adds IPC latency. FastAPI provides asynchronous ASGI streaming for Server-Sent Events out-of-the-box.

Frontend Layer	React + Vite + Tailwind	Streamlit / Gradio	Streamlit/Gradio look like internal research demos. React allows custom responsive design tokens, live visual step-tracing, persistent layouts, and production-grade software presentation.
RAG Algorithm	Corrective RAG (CRAG)	Naive / Vanilla RAG	Naive RAG blindly injects top-k chunks into the prompt, risking hallucinations when chunks are noisy. CRAG grades chunks first and triggers query rewriting and web search if documents lack coverage.
Embedding Model	Local ONNX MiniLM	OpenAI text-embedding-3	Runs locally on CPU in milliseconds with \$0.00 API costs, 100% offline capability, and zero external data transmission (air-gapped privacy).

Streaming Method	Server-Sent Events (SSE)	WebSockets / Polling	WebSockets require complex duplex connection state. Polling wastes requests. SSE runs over standard HTTP, auto-reconnects, and streams text & step events natively.
Judge Experience	1-Click Judge Mode	Mandatory User Signup	Judges only have 3-5 minutes. Forcing account creation or email verification kills demos. Judge Mode bypasses external dependencies with guaranteed uptime.

4. High-Impact AI Vocabulary (Memorize for Presentation)

1. Hallucination Mitigation

Constraining generative output strictly to verified context chunks using negative prompt boundaries (*"Do not extrapolate beyond provided text"*).

2. Corrective RAG (CRAG)

An agentic retrieval architecture that incorporates an active evaluator node to self-correct retrieval gaps prior to answer synthesis.

3. Dual-Model Architecture

Partitioning tasks: a sub-second lightweight model (20B) handles parallel document grading, while a 70B/120B model performs final constrained reasoning.

4. Grounded Provenance & Attribution

Attaching explicit document filenames and page numbers to claims, establishing an immutable audit trail for evaluators.

5. StateGraph Orchestration

Modeling cognitive agent flows as cyclical, state-preserving directed graphs (via LangGraph) with conditional branching and self-healing edges.

6. Normalized Cosine Vector Distance

Measuring angular difference between embedding vectors, ensuring chunk length does not disproportionately bias retrieval relevance.

5. 12-Hour Hackathon Execution Timeline (Tonight)

Time Window	Primary Objective	Action Checklist
Hours 0 – 1	PS Ingestion & Scope Fit	Read PS carefully. Identify target persona and constraints. Copy PDFs into <code>backend/research_papers/</code> . Set brand tokens in <code>DESIGN_BRIEF.md</code> .
Hours 1 – 3	Vector Ingestion & Chunking	Clear <code>backend/rag_db/</code> to rebuild index. Run ingestion script with 4,000-char chunks and 300 overlap. Verify retrieval via <code>test_server.py</code> .
Hours 3 – 6	Domain Prompt & Logic	Tune <code>PROMPT_QA</code> in <code>crag_app.py</code> with domain-specific rules (tables, structured outputs). Test API endpoints via <code>http://localhost:8000/docs</code> .
Hours 6 – 9	Frontend Customization	Update <code>DashboardPage.tsx</code> with domain labels and sample queries. Ensure all text $\geq 14\text{px}$ and body $\geq 16\text{px}$. Run <code>npm run design:check</code> .

Hours 9 – 11	End-to-End Verification	Test live SSE stream on 3 demo queries (Direct RAG, Corrective Search, Offline simulation). Run <code>npm run build</code> and push to GitHub.
Hours 11 – 12	Pitch Rehearsal & Polishing	Rehearse the 3-minute pitch. Lock down who speaks for each slide. Verify 1-Click Judge Mode works with Wi-Fi disconnected.

6. 3-Minute Winning Pitch Script

0:00 – 0:45 | The Hook & Problem Statement

"Good evening judges. When teams evaluate technical problem statements or regulatory briefs, standard AI chatbots fail: they hallucinate, invent false citations, and leave users with zero auditability. In real-world operations, unverified advice leads to compliance failure."

0:45 – 1:30 | The Core Solution & Live Demo

"To solve this, we engineered an Agentic Corrective RAG (CRAG) platform. Notice as we submit this research query on our screen: in real time, our engine queries the local ChromaDB vector space, executes parallel batch relevance grading across chunks, and synthesizes a verified answer with exact page citations."

1:30 – 2:15 | The Self-Correcting Edge (The 'Wow' Factor)

"What sets our system apart is self-correction. If our internal documents lack sufficient evidence, the agent automatically identifies the gap, rewrites the query, searches the live web, and synthesizes augmented context—marked by the live 'CORRECTIVE PATH' badge."

2:15 – 3:00 | Engineering Excellence & Resilience

"Our stack is decoupled for enterprise scale: React on the frontend with Server-Sent Events, and FastAPI with LangGraph on the backend. Everything runs on local ONNX embeddings for data privacy and zero API overhead, backed by a 1-click Judge Mode

for zero-downtime evaluation."

7. The 5 Hardest Judge Questions & Winning Answers

Q1: "Why shouldn't we just upload these PDFs to ChatGPT / NotebookLM?"

"Consumer tools lack enterprise data privacy, deterministic API integration, and verifiable provenance. Our solution runs local ONNX embeddings where sensitive data never leaves the premises, validates chunk relevance through a dual-model gatekeeper, and integrates into business pipelines via REST/SSE."

Q2: "What happens if ChromaDB retrieves completely irrelevant chunks?"

"That is the exact problem CRAG solves. Naive RAG passes bad chunks to the LLM and hallucinates. Our LangGraph pipeline has an active evaluator node. If less than 50% of chunks are relevant, it rejects the context, rewrites the query, and searches live web sources to self-correct."

Q3: "Why did you use ONNX MiniLM instead of OpenAI embeddings?"

"Three critical advantages: **zero API cost**, **sub-millisecond local latency**, and **air-gapped privacy**. Internal problem documents are processed entirely in-memory on CPU without transmitting sensitive data to third-party endpoints."

Q4: "How does the system handle high query concurrency during peak load?"

"Our grading node executes parallel batch inference using a

lightweight model (Groq 20B), completing chunk evaluations in under 700ms. The FastAPI layer is fully asynchronous (ASGI), allowing hundreds of concurrent SSE streams without thread starvation."

Q5: "How does your architecture scale if we add 10,000 documents?"

"ChromaDB uses HNSW (Hierarchical Navigable Small World) index graphs with logarithmic search complexity $O(\log N)$. Because our LangGraph pipeline is decoupled, we can seamlessly point the retriever to enterprise vector engines like Milvus or Pinecone with zero changes to agent logic."